

SHIV NADAR UNIVERSITY, CHENNAI

Department of Computer Science and Engineering

DEEP LEARNING LABORATORY

Experiment 1

Implementation of a Single Layer Perceptron for Binary
Classification

Name: Sukesh S

Roll No: 24011101111

Programme: B.Tech Computer Science and Engineering

Semester: V

Academic Year: 2026–2027

Department of Computer Science and Engineering
Shiv Nadar University Chennai

1 Objective

The objectives of this experiment are:

- To understand the architecture and working principle of a Single Layer Perceptron.
- To implement the Perceptron Learning Algorithm from scratch using Python.
- To classify the Banknote Authentication dataset into genuine and forged classes.
- To evaluate the classifier using Accuracy, Precision, Recall and F1-score.
- To compare the custom implementation with the Scikit-Learn implementation.
- To analyze the effect of different learning rates on convergence.

2 Background Theory

AI has improved and changed rapidly over the few decades with machine learning and deep learning has become very important. ANNs are actually inspired by the biological neurons present in human brain. The simplest neural network is called the perceptron.

A single layer perceptron can only solve linearly separable binary classifications. Even with this it is the basis for modern neural networks.

Each of the neuron has input features and multiplied by corresponding weight adds a bias and passes through an activation function.

Mathematically,

$$z = w_1x_1 + w_2x_2 + \cdots + w_nx_n + b$$

or equivalently,

$$z = \mathbf{w}^T \mathbf{x} + b$$

where

- $\mathbf{x}$ is the input vector
- $\mathbf{w}$ is the weight vector
- b is the bias
- z is the linear combination

The perceptron predicts the class based on the sign of z .

3 Perceptron Learning Rule

Initially, the weights and bias are initialized to small random values or zeros.

For every training sample,

$$\hat{y} = f(z)$$

where

$$f(z) = \begin{cases} 1, & z \geq 0 \\ 0, & z < 0 \end{cases}$$

The prediction error is

$$e = y - \hat{y}$$

The parameters are updated using

$$w_i = w_i + \eta e x_i$$

$$b = b + \eta e$$

where

- η = learning rate
- e = prediction error
- x_i = i th feature

Training continues until convergence or until the maximum number of epochs is reached.

4 Activation Functions

Activation functions introduce non-linearity into neural networks.

The Perceptron uses the Step activation function.

4.1 Step Function

$$f(x) = \begin{cases} 1, & x \geq 0 \\ 0, & x < 0 \end{cases}$$

Advantages:

- Simple
- Computationally inexpensive
- Suitable for binary classification

Disadvantages:

- Non-differentiable
- Zero gradient
- Cannot be used with backpropagation

4.2 Comparison of Activation Functions

Table 1: Comparison of Common Activation Functions

Activation	Range	Differentiable	Applications
Step	$\{0,1\}$	No	Perceptron
Sigmoid	$(0,1)$	Yes	Binary Output
Tanh	$(-1,1)$	Yes	Hidden Layers
ReLU	$[0,\infty)$	Yes	Deep Networks
Leaky ReLU	$(-\infty,\infty)$	Yes	Avoid Dead Neurons
Softmax	$(0,1)$	Yes	Multi-class Classification

5 Dataset Description

The Banknote Authentication dataset is obtained from the UCI Machine Learning Repository.

It contains statistical features extracted from wavelet transformed banknote images.

Table 2: Dataset Summary

Dataset	Banknote Authentication
Number of Samples	1372
Training Samples	1097
Testing Samples	275
Number of Features	4
Number of Classes	2
Missing Values	None
Normalization	StandardScaler

The four numerical features are:

1. Variance
2. Skewness
3. Curtosis
4. Entropy

Target labels:

- 0 = Genuine Note
- 1 = Forged Note

6 Experimental Procedure

The following procedure was carried out during the experiment.

1. Load the Banknote Authentication dataset.
2. Perform EDA.
3. Check for missing values.
4. Normalise using StandardScaler.
5. Split the dataset into training and testing sets.
6. Start with initialization of the perceptron weights and bias.
7. Train the perceptron using the learning rule.
8. Record the evolution of weights and bias.
9. Evaluate the classifier using Accuracy, Precision, Recall and F1-score.
10. Compare the custom implementation with Scikit-Learn.
11. Repeat training for multiple learning rates.

7 Source Code

The complete implementation is available at:

<https://github.com/ssukesh182/DeeplearningLab>

8 Experimental Results

8.1 Training Summary

Table 3: Training Summary

Parameter	Value
Dataset Size	1372
Training Samples	1097
Testing Samples	275
Epochs	100
Learning Rate	0.001
Best Learning Rate	0.0001
Normalization	StandardScaler
Classes	2
Features	4

8.2 Performance Metrics

Table 4: Custom Perceptron Performance

Metric	Value
Accuracy	99.27%
Precision	100.00%
Recall	98.36%
F1 Score	99.17%

8.3 Learning Rate Comparison

Table 5: Learning Rate Analysis

Learning Rate	Accuracy
0.0001	99.27%
0.001	98.55%
0.01	98.91%
0.10	98.55%
0.50	98.91%
1.00	98.55%

8.4 Comparison with Scikit-Learn

Table 6: Custom Perceptron vs Scikit-Learn

Metric	Custom	Scikit-Learn
Accuracy	99.27%	100.00%
Precision	100.00%	100.00%
Recall	98.36%	100.00%
F1 Score	99.17%	100.00%

9 Generated Plots

9.1 Class Distribution

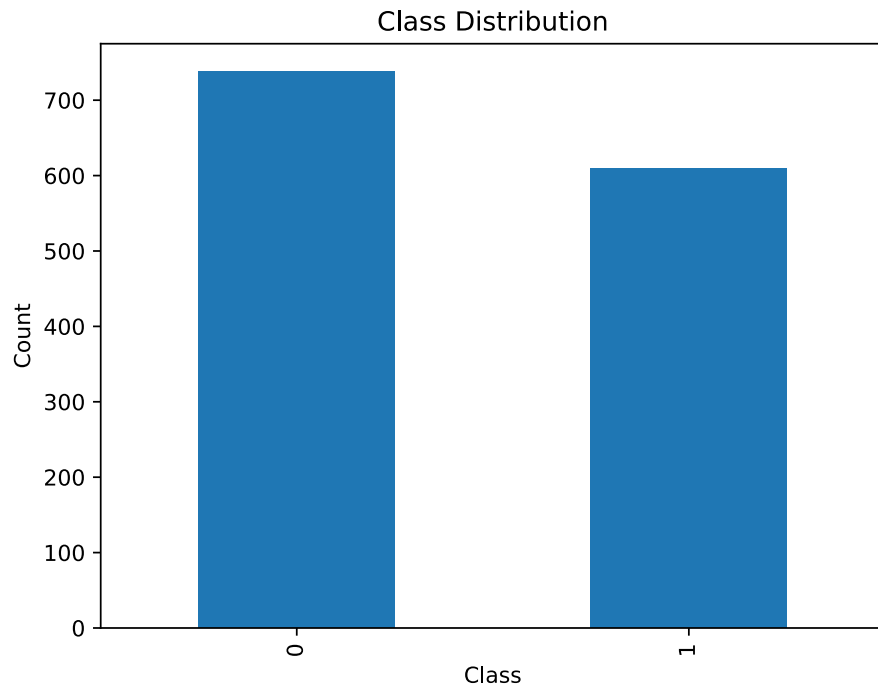


Figure 1: Class Distribution

The dataset contains an almost equal number of genuine and forged banknotes. This balanced distribution minimizes class imbalance and enables fair model training.

9.2 Feature Box Plots

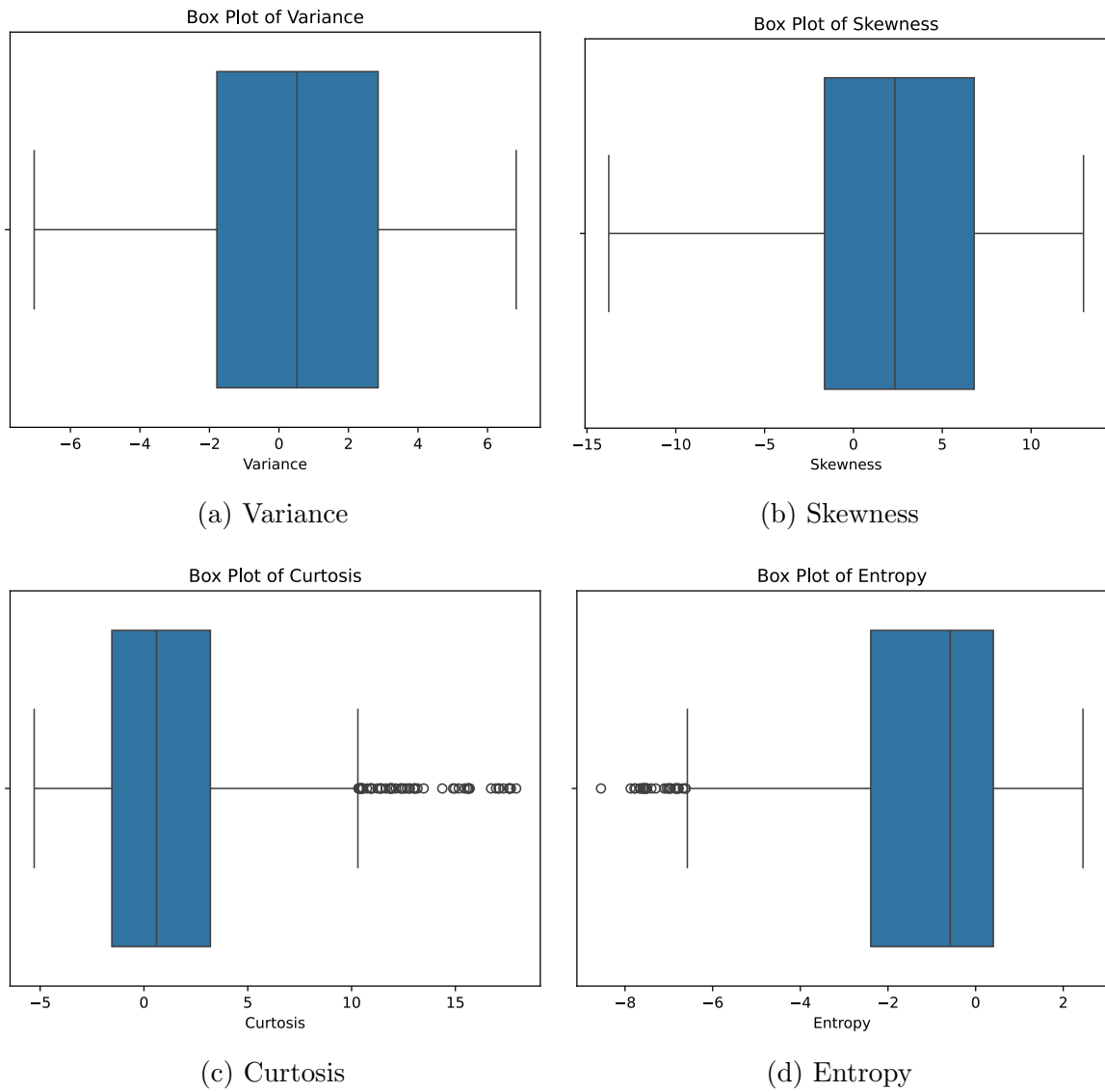


Figure 2: Box Plots of Features

The box plots illustrate the spread, median and outliers for each feature. .

9.3 Feature Histograms

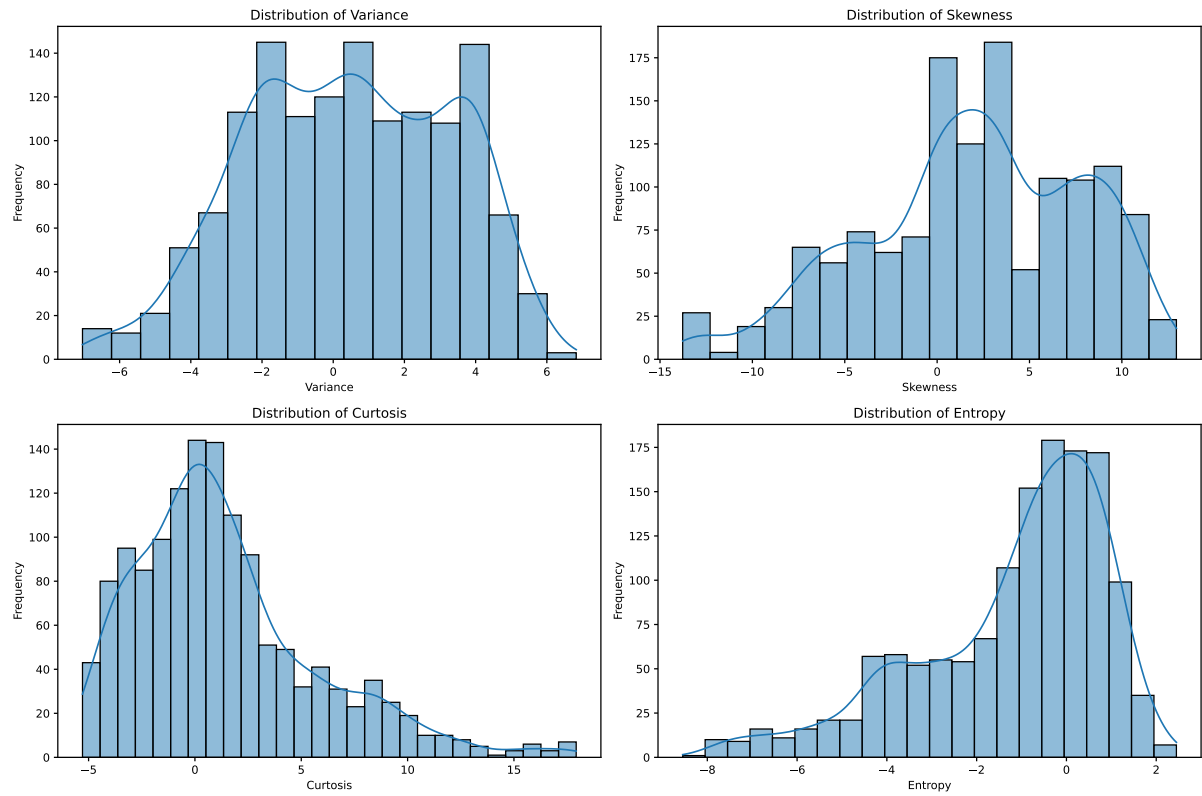


Figure 3: Feature Distributions

The histogram provided us info that each has their own distribution.

9.4 Pair Plot

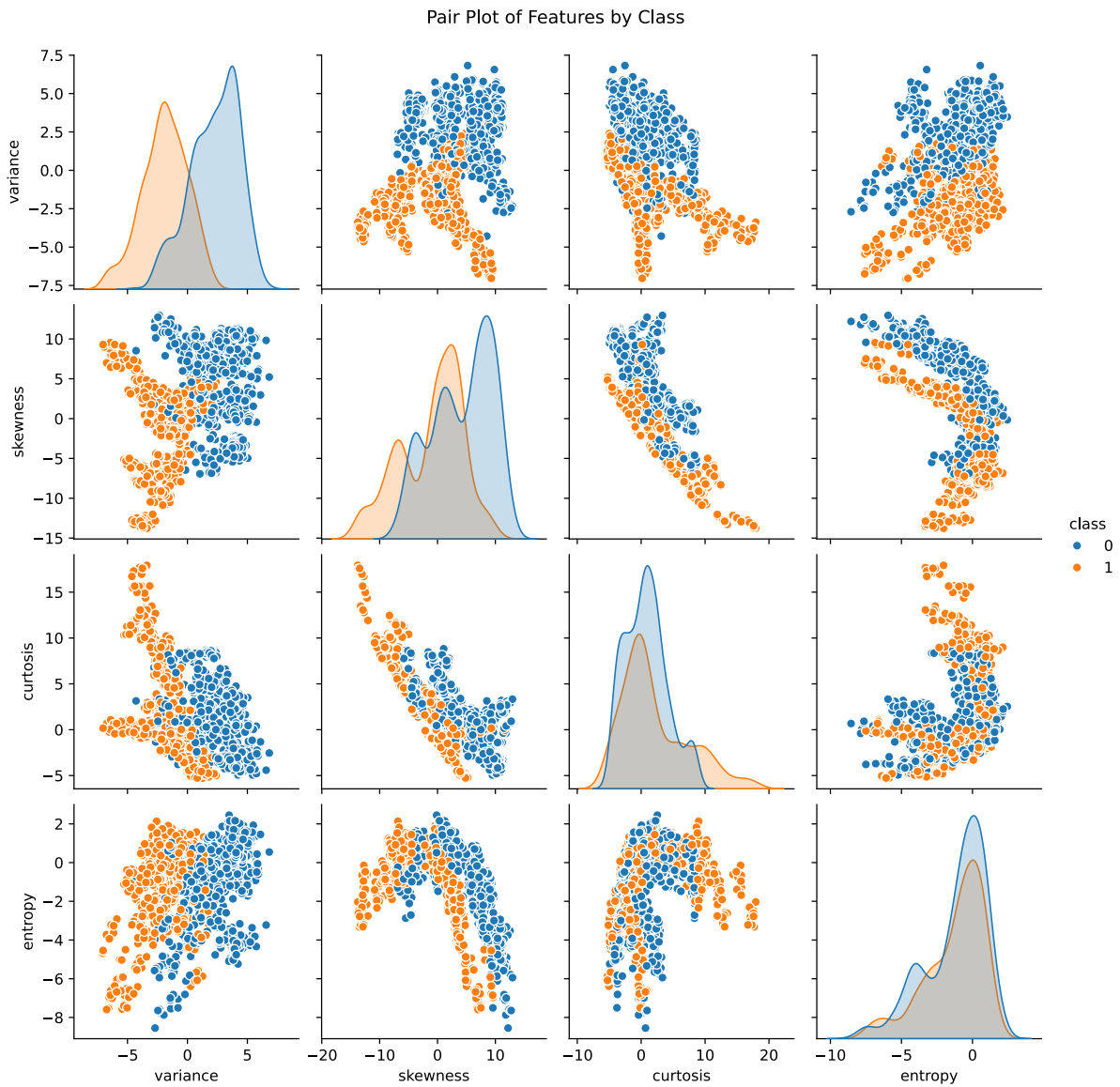


Figure 4: Pair Plot of Features

Pairplot shows us different possible separations possible showing it is linearly separable.

9.5 Correlation Matrix

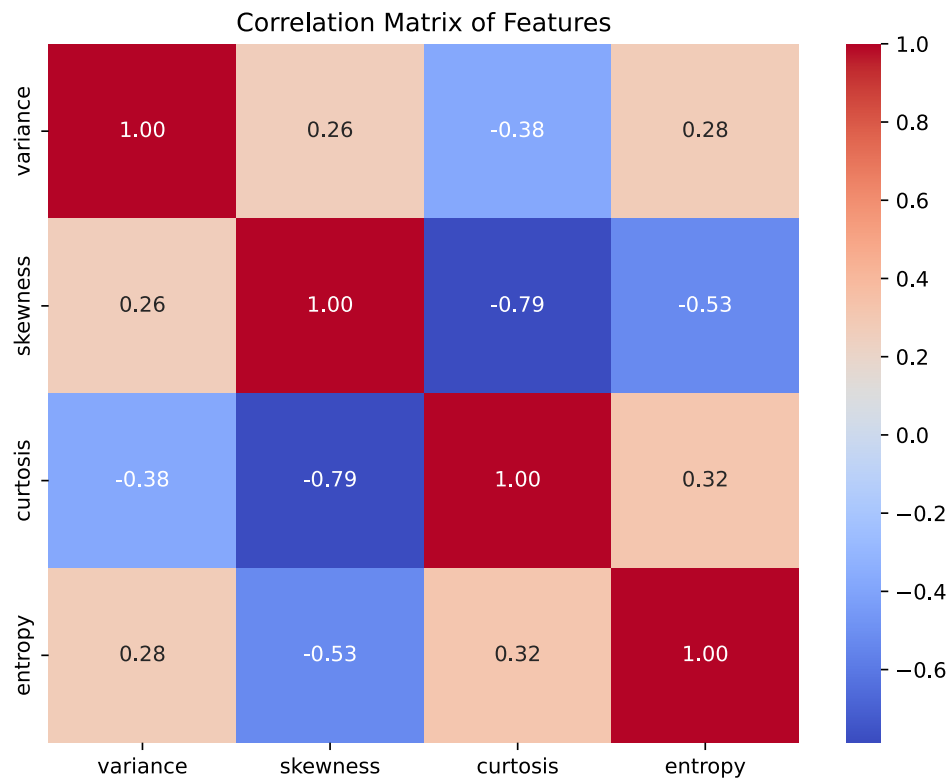


Figure 5: Correlation Matrix

The result from the heatmap shows strong negative correlation between Skewness and curtosis while other featuers have less correlations for both positive and negative.

9.6 Scatter Plot: Variance vs Skewness

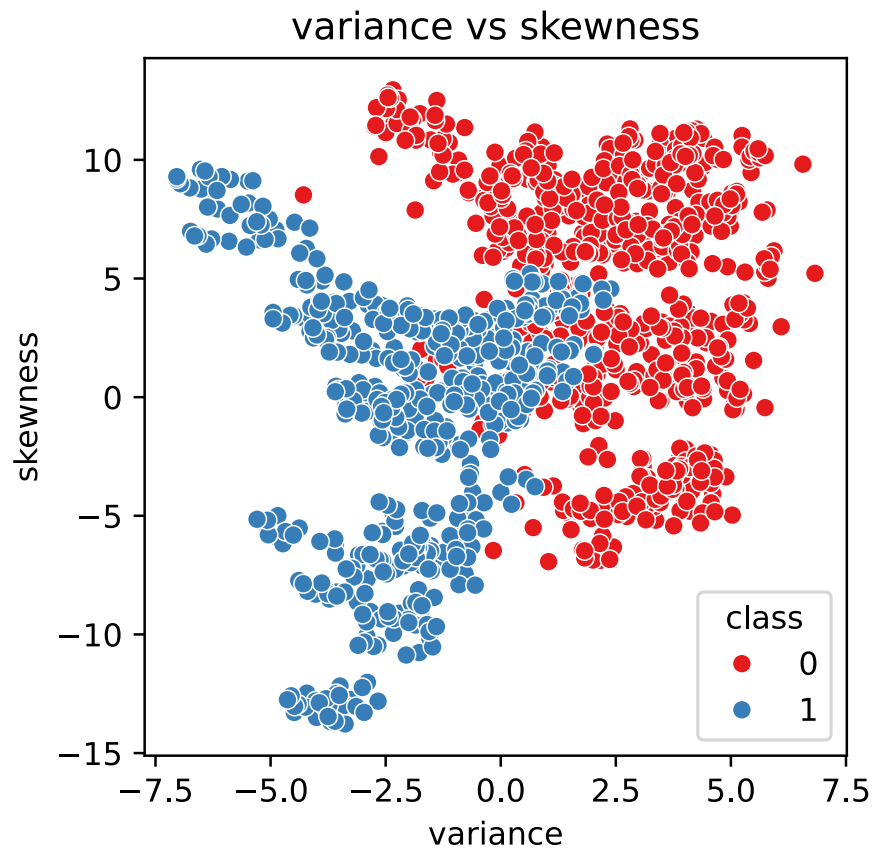


Figure 6: Variance vs Skewness

Significantly contributes to binary classification as it splits the classes strongly.

9.7 Scatter Plot: Variance vs Kurtosis

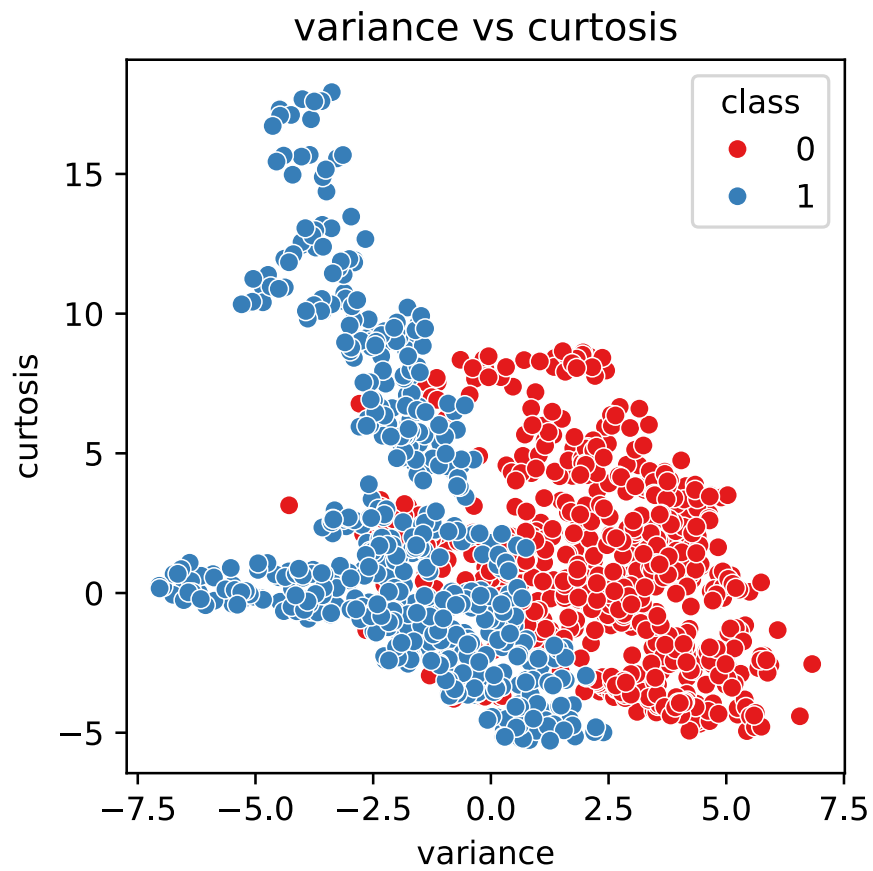


Figure 7: Variance vs Kurtosis

A noticeable separation exists between the two classes, showing impact to separate based on these features.

9.8 Scatter Plot: Variance vs Entropy

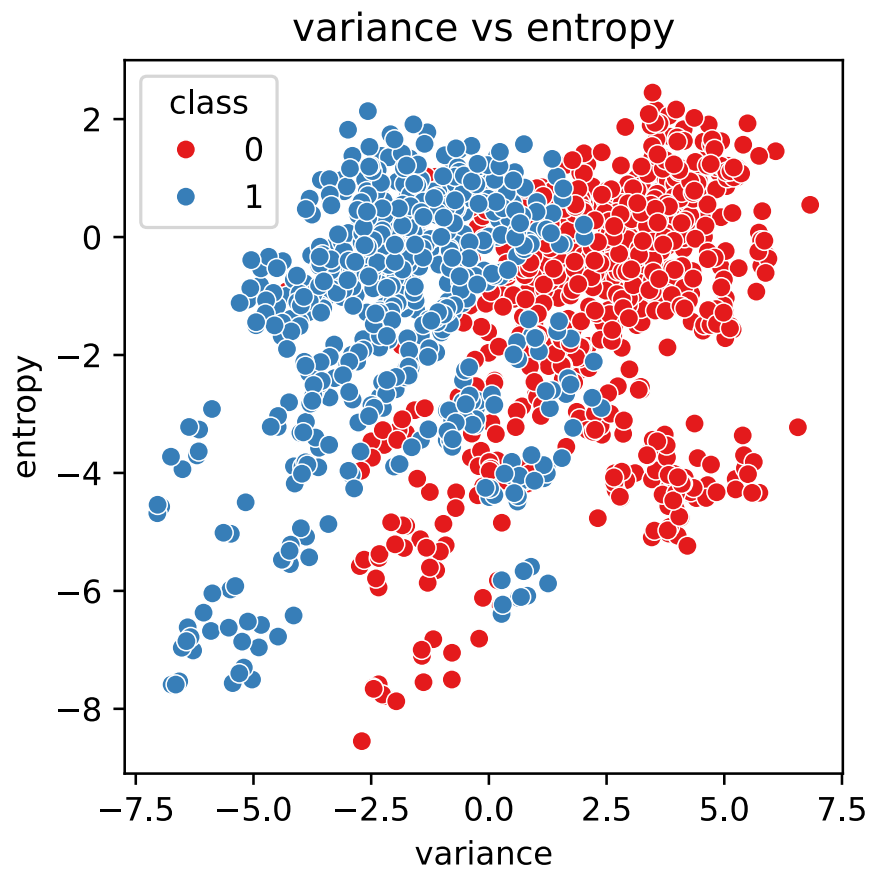


Figure 8: Variance vs Entropy

Not much information as they overlap.

9.9 Scatter Plot: Skewness vs Kurtosis

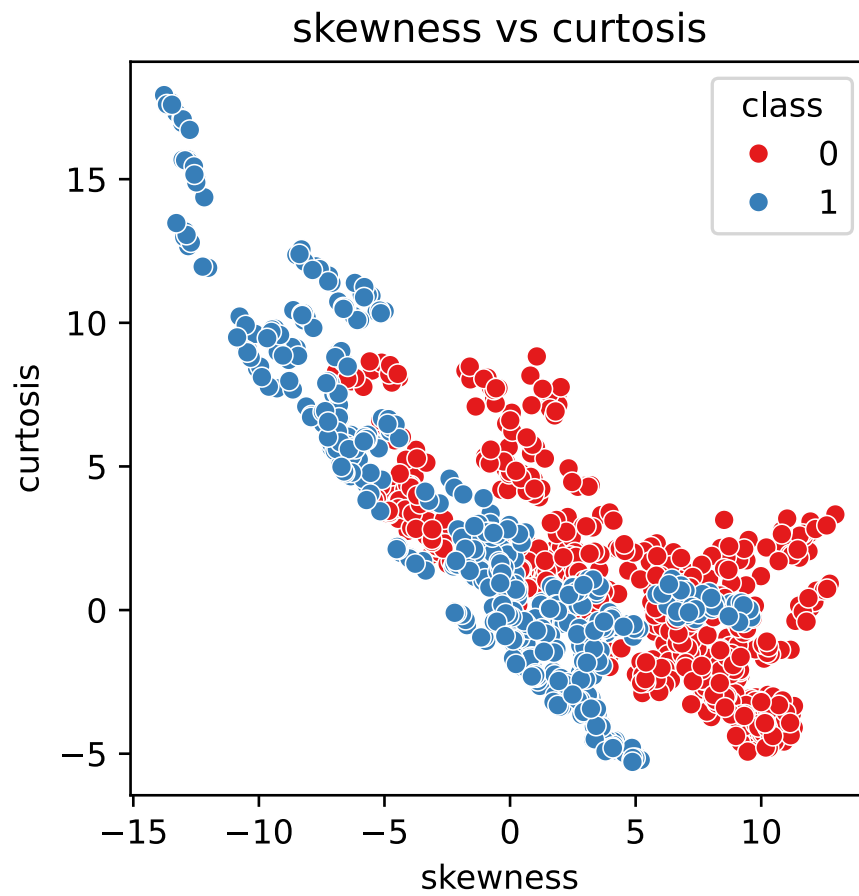


Figure 9: Skewness vs Kurtosis

Show strong negative correlation.

9.10 Scatter Plot: Skewness vs Entropy

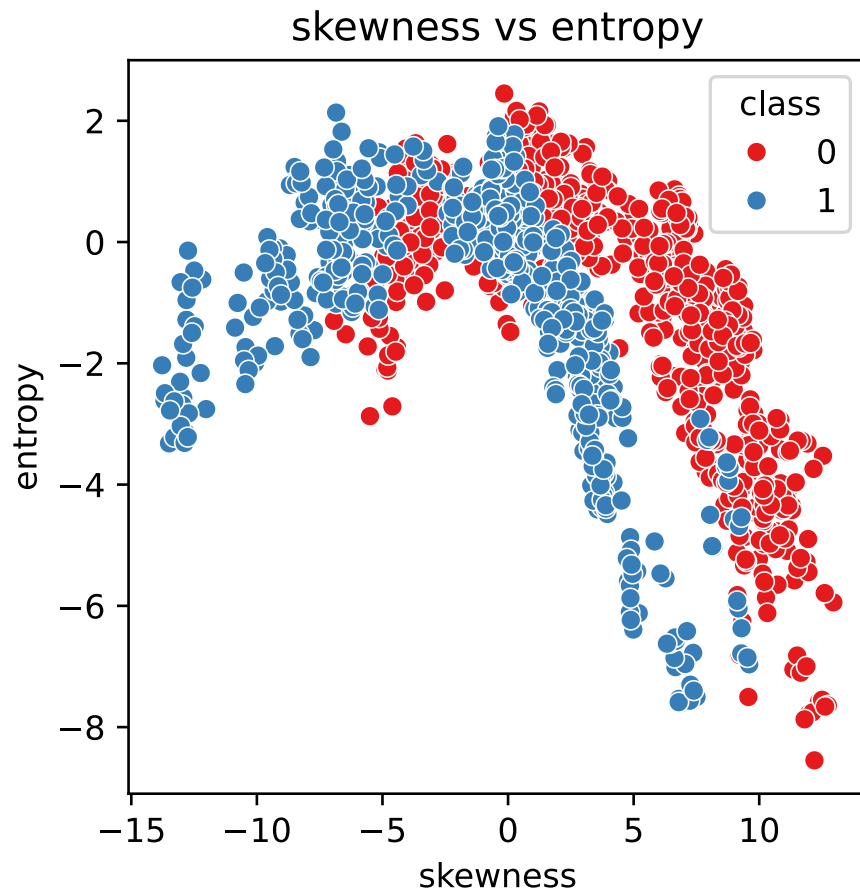


Figure 10: Skewness vs Entropy

Better separation can be achieved through this as there is visual separation of the two classes.

9.11 Scatter Plot: Curtosis vs Entropy

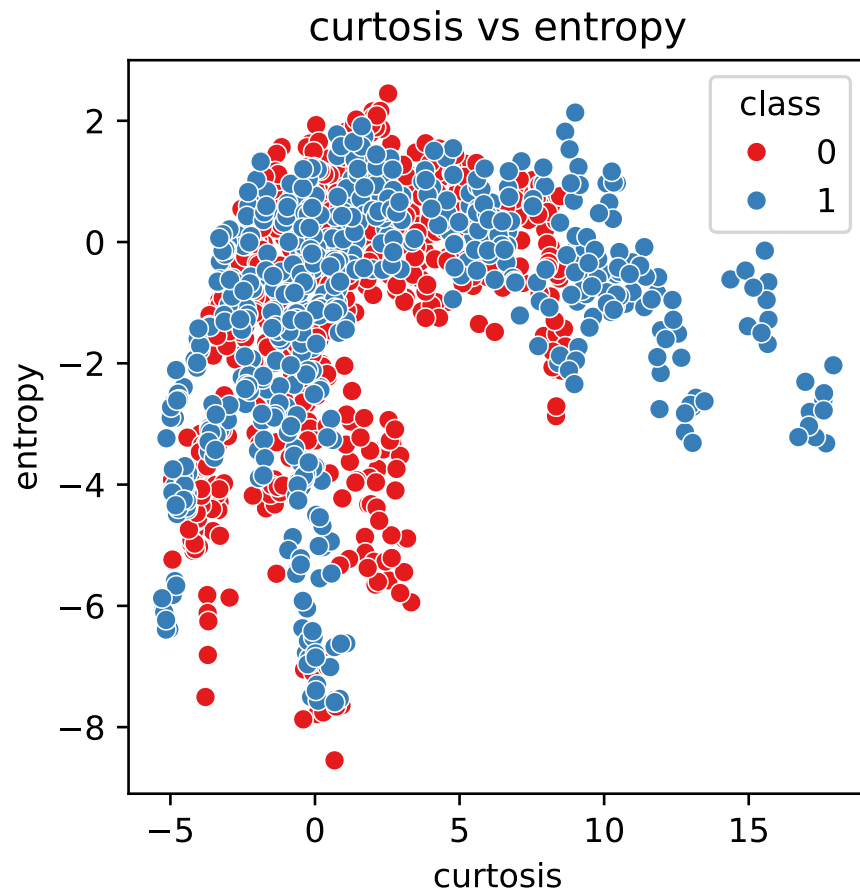


Figure 11: Curtosis vs Entropy

Distinguishable clusters corresponding to the two output classes.

9.12 Weight Evolution

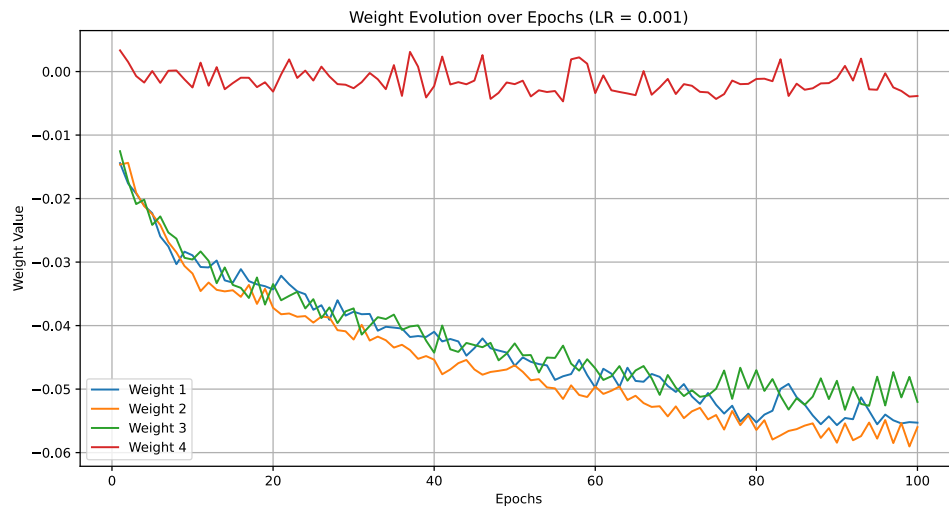


Figure 12: Weight Evolution during Training

The weights stabilises as training progress, indicating the convergence.

9.13 Bias Evolution

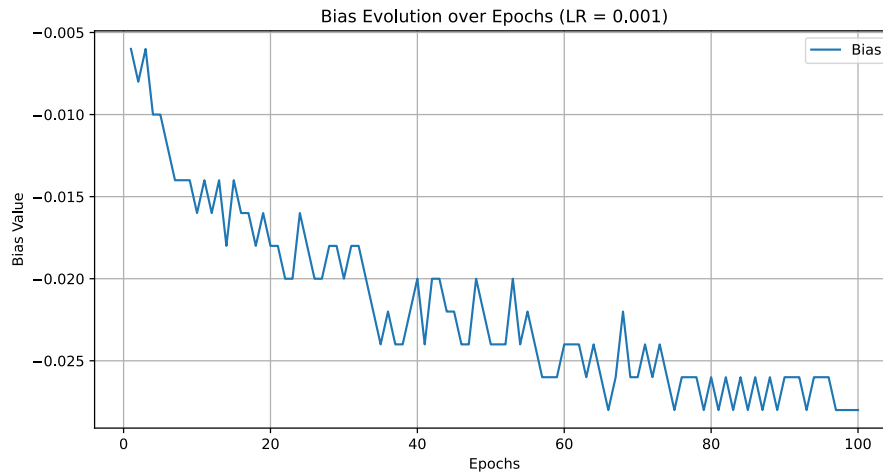


Figure 13: Bias Evolution during Training

The bias value converges smooth as epochs increase.

9.14 Learning Rate Analysis

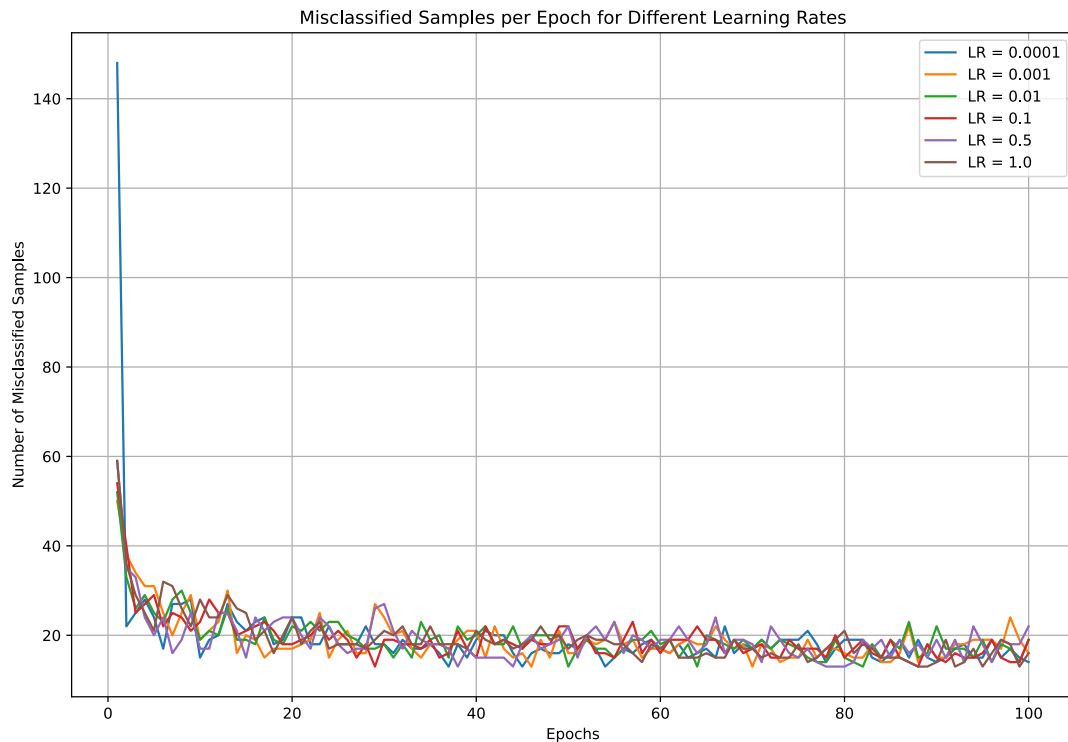


Figure 14: Misclassified Samples for Different Learning Rates

Low learning rates produce slow and smooth convergence whereas large learning rates show a sudden fluctuations before proper results.

10 Discussion

Our implemented perceptron has achieved very good classification performance of 99.27 percent accuracy which is really high. This shows the quality of the dataset which is almost linearly separable so the single layer perceptron learned a good decision boundary. Normalising the features improved stability and proper weight updates. The comparison with scikit learn showed the implemented model is closer to perfect.

11 Additional Questions

1. Compare Step and Sigmoid Activation Functions

Step function is only used for getting binary outputs and can only be used in simple neurons where the sigmoid is continuous and differentiable and also can work well with gradient.

2. Why is the Step Function not used in Deep Learning?

Since the Step function has non differentiable parts and always result in 0. There is no possibility for a backprop learning and cannot go through backward pass making it not suitable in deep learning.

3. Comparison with Scikit-Learn

The scikit learn performed better than our custom built model. As it is highly optimized than our model.

4. Effect of Learning Rate

Smaller learning rate give slower and stable convergence where as when the learning rate is high the convergence is fast and it needs additional iterations just for stabilisation

5. Why can't a Single Layer Perceptron solve XOR?

A single layer can't solve XOR problem because it cannot identify the decision boundary of non linear problem and XOR is a non linear problem.

6. Effect of Feature Normalization

It is very useful as it scales the input features on one range which is comparable to provide numeric stability and good weight updates in the future.

12 Conclusion

This experiment showed and demonstrated the single layer perceptron from the bottom on how its is used for binary classification. The model has achieved 99.27 percent accuracy on the dataset: Banknote Authentication and produced competitive results with the scikit learn's single layer perceptron. The experiment focused on learning rate selection, feature normalisation and EDA techniques to give high classification performance.

13 Perceptron-Based Implementation of Logic Gates

13.1 Theory

Perceptron is the simple form of ANN. It was introduced by Rosenblatt it comprised McCulloch Pitts neuron. Where weight was an extra feature. Given an input vector

$\mathbf{x} = (x_1, x_2, \dots, x_n)$, the neuron computes a weighted sum

$$z = \mathbf{w}^\top \mathbf{x} + b = \sum_{i=1}^n w_i x_i + b \quad (1)$$

and gives it to the next step activation function

$$f(z) = \begin{cases} 1, & z \geq 0 \\ 0, & z < 0 \end{cases} \quad (2)$$

and gives an output $\hat{y} = f(z)$.

Perceptron learning rule is used for prediction of weights and bias. For each training sample $(\mathbf{x}_i, y_i)$, the prediction error is $e = y_i - \hat{y}_i$, and if $e \neq 0$ the updates are

$$\mathbf{w} \leftarrow \mathbf{w} + \eta e \mathbf{x}_i \quad (3)$$

$$b \leftarrow b + \eta e \quad (4)$$

where η is the learning rate (taken as $\eta = 1$ here). Training proceeds epoch by epoch until zero error at which point the decision boundary $\mathbf{w}^\top \mathbf{x} + b = 0$ correctly. As single perceptron can only have a linear decision boundary it can only separate other linear functions and will not work with non linear functions. It works good on AND, OR, and NOT, but cannot learn XOR.

Weights were initialized to zero ($\mathbf{w} = \mathbf{0}$, $b = 0$) and the network was trained for a maximum of 10 epochs on each gate's truth table.

13.2 Results

Table 7 Learned parameters and weight updates summarised

Table 7: Summary

Gate	Epochs to Converge	Total Updates	Final Weights ($\mathbf{w}$)	Final Bias (b)
AND	5	11	[2.0, 1.0]	-3
OR	3	5	[1.0, 1.0]	-1
NOT	2	2	[-1.0]	0

13.2.1 AND Gate

Table 8: Weight updates during perceptron training for the AND gate

Update	Epoch	Input (x_1, x_2)	Target	Predicted	Error	Updated $\mathbf{w}$	Updated b
1	1	(0,0)	0	1	-1	(0.0, 0.0)	-1
2	1	(1,1)	1	0	+1	(1.0, 1.0)	0
3	2	(0,0)	0	1	-1	(1.0, 1.0)	-1
4	2	(0,1)	0	1	-1	(1.0, 0.0)	-2
5	2	(1,1)	1	0	+1	(2.0, 1.0)	-1
6	3	(0,1)	0	1	-1	(2.0, 0.0)	-2
7	3	(1,0)	0	1	-1	(1.0, 0.0)	-3
8	3	(1,1)	1	0	+1	(2.0, 1.0)	-2
9	4	(1,0)	0	1	-1	(1.0, 1.0)	-3
10	4	(1,1)	1	0	+1	(2.0, 2.0)	-2
11	5	(0,1)	0	1	-1	(2.0, 1.0)	-3

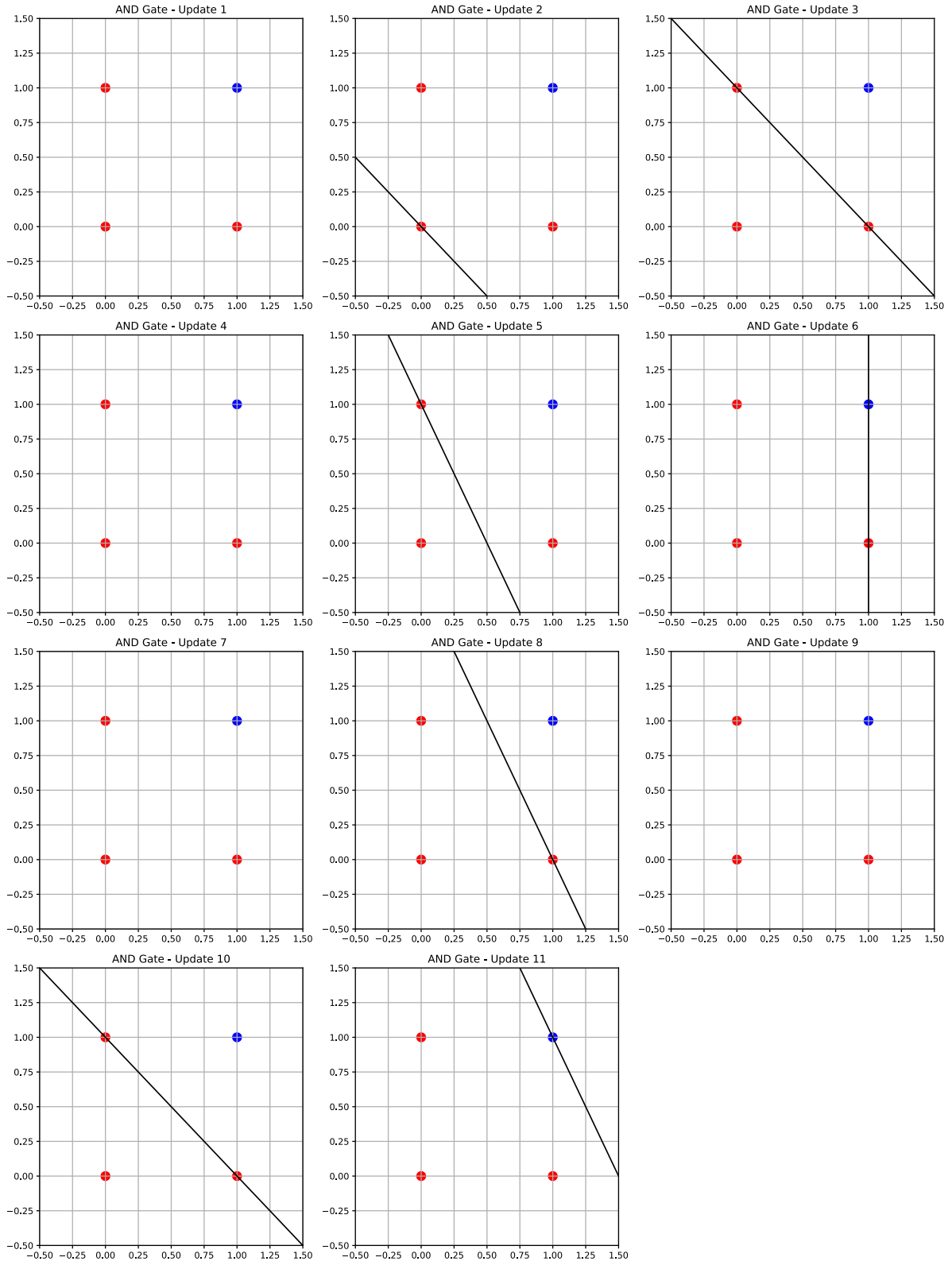


Figure 15: Change in OR gate over epochs

13.2.2 OR Gate

Table 9: Update weights during perceptron for OR

Update	Epoch	Input (x_1, x_2)	Target	Predicted	Error	Updated $\mathbf{w}$	Updated b
1	1	(0,0)	0	1	-1	(0.0, 0.0)	-1
2	1	(0,1)	1	0	+1	(0.0, 1.0)	0
3	2	(0,0)	0	1	-1	(0.0, 1.0)	-1
4	2	(1,0)	1	0	+1	(1.0, 1.0)	0
5	3	(0,0)	0	1	-1	(1.0, 1.0)	-1

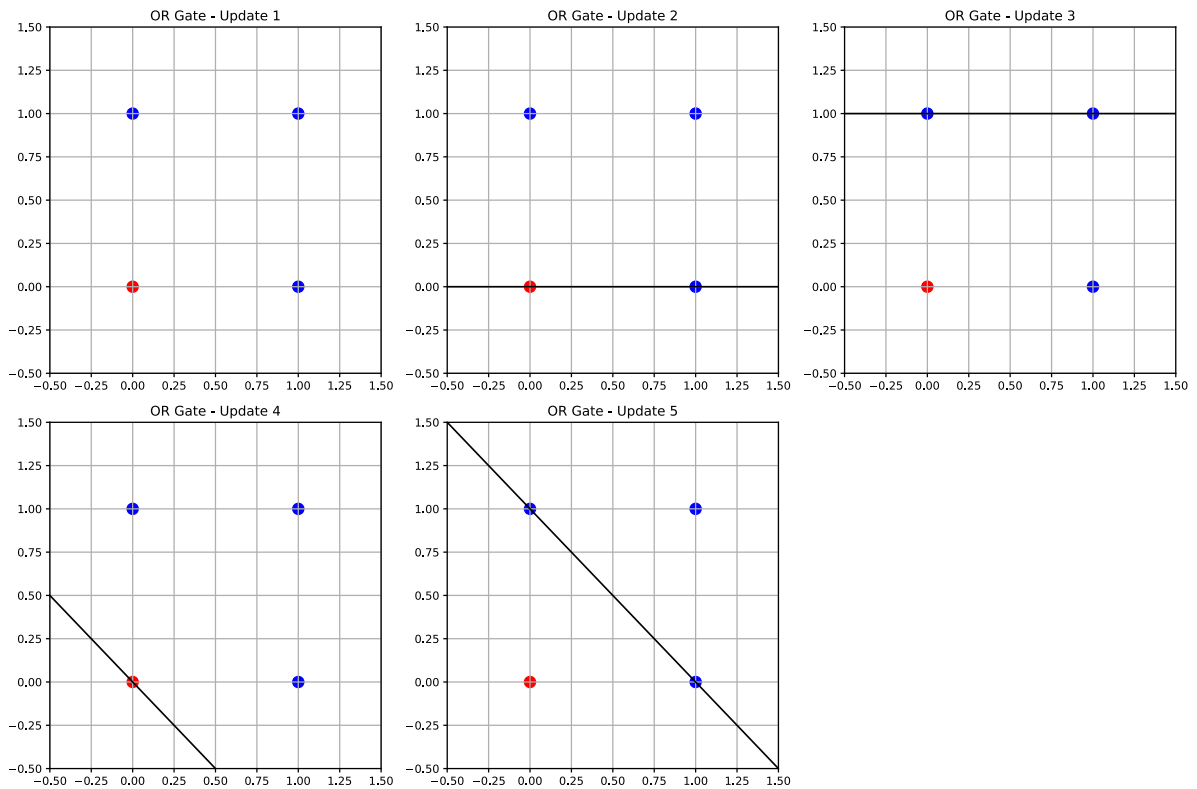


Figure 16: Change in OR gate's decision boundary

13.2.3 NOT Gate

Table 10: Weight updates during perceptron training for the NOT gate

Update	Epoch	Input (x_1)	Target	Predicted	Error	Updated w	Updated b
1	1	(1)	0	1	-1	-1.0	-1
2	2	(0)	1	0	+1	-1.0	0

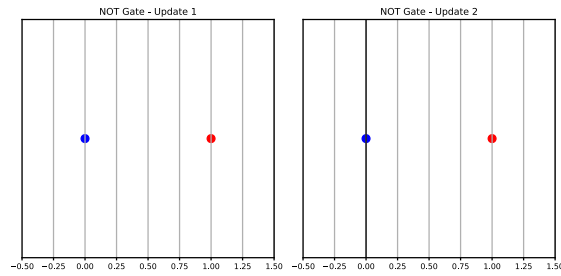


Figure 17: Change of the decision boundary for Not gate

13.3 Observations

In every case the perceptron converge with few epoch since AND, OR and NOT are linearly separable functions all of them require only a straight line to be the decision boundary. The And gate took the longest time and updates because only one of the four input combination is true so as to shift the boundary properly. The OR gate converges faster as 3 of the conditions are true. The not gate having only 2d inputs converges in just 2 updates

14 References

1. Frank Rosenblatt, *The Perceptron: A Probabilistic Model for Information Storage and Organization in the Brain*, 1958.
2. Ian Goodfellow, Yoshua Bengio and Aaron Courville, *Deep Learning*, MIT Press.
3. Simon Haykin, *Neural Networks and Learning Machines*.
4. UCI Machine Learning Repository — Banknote Authentication Dataset.
5. Scikit-Learn Documentation.